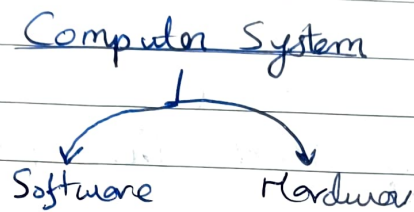


Introduction to Computers.

Computer - A computer is a machine that →

- ① Inputs data from the user
- ② Stores & processes that data
- ③ outputs the results of the process



Hardware

① Input devices - Takes input from user and converts user readable language to binary language.

② Output devices - Provides output after operation and converts binary languages to user readable language.

CPU (Central Processing Unit)

① ALU (Arithmetic logic unit)

It performs arithmetic & logical calculations based on the instructions from the control unit.

② CU (Control Unit)

It controls the data processing operations.

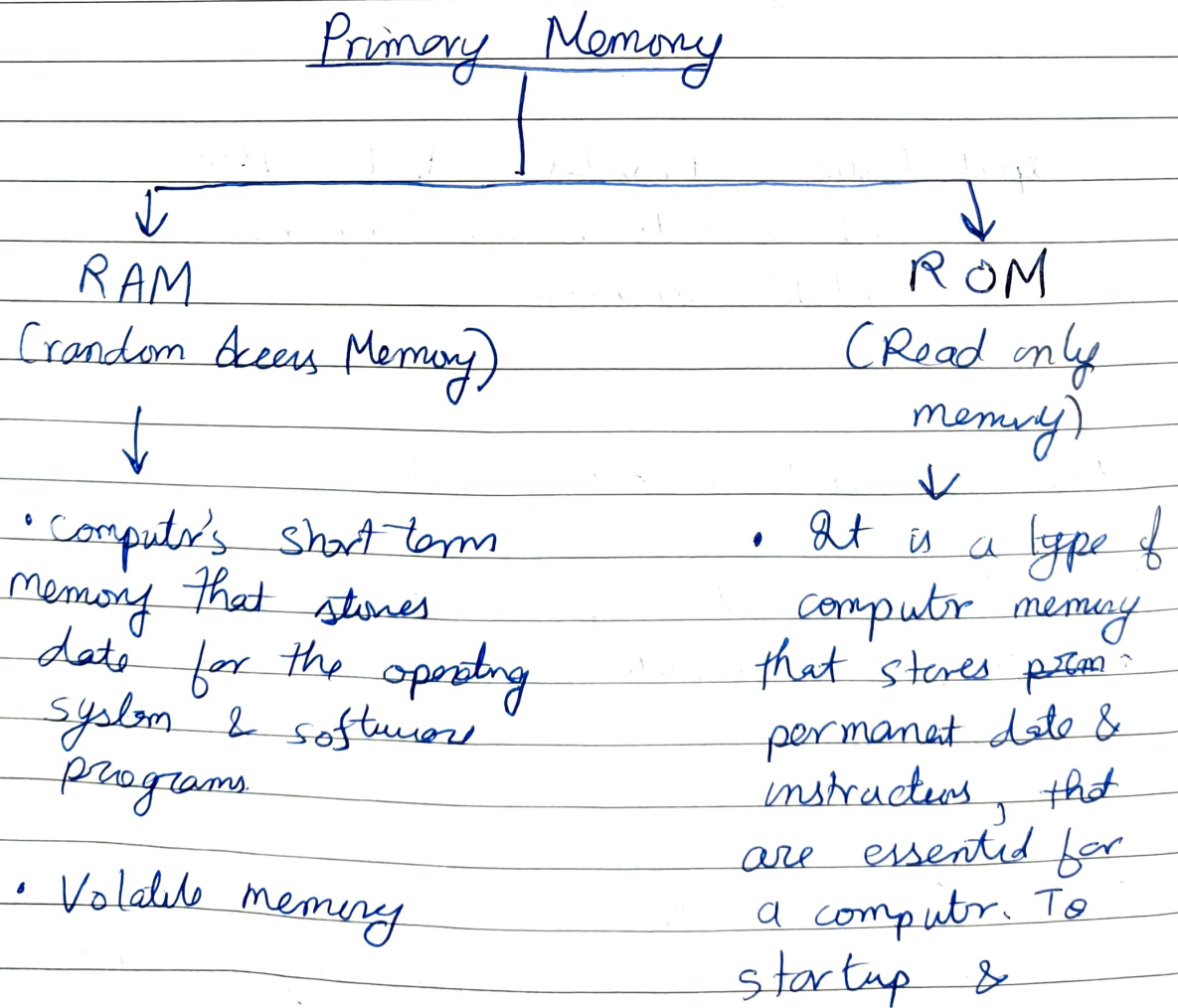
Page _____

③ Register: A small storage unit built into the CPU for temporary storage and manipulation of data during the execution of instructions.

Memory

① Primary memory: It is a section of a computer's memory that the CPU can directly access.

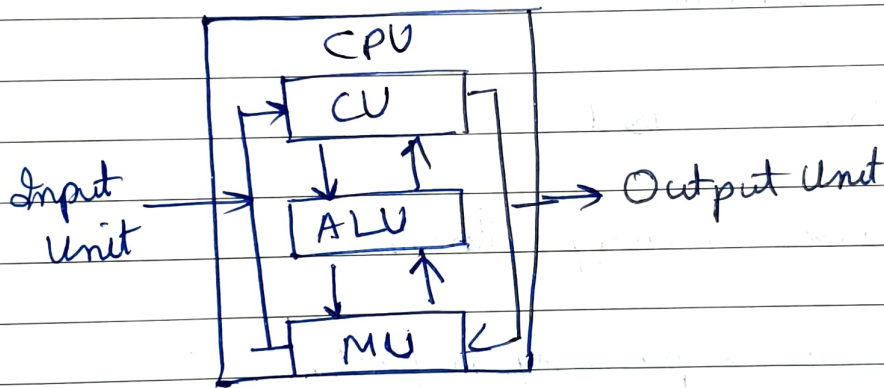
Primary memory is volatile meaning that the contents are lost when the computer's power supply is turned off or interrupted.



⑪ Secondary Memory → It is a computer memory that is non-volatile, persistent and not immediately accessible by a computer or processor. It allows users to store data & information that can be retrieved & used.

⑫ Cache memory: Placed b/w RAM & CPU, it increases the speed of processing. By doing so it improves throughput of the machine.

Characteristics



Characteristics of C Lang

Compiler - Translates high level language to machine language.

① Portable → can be run in different machine ~~for~~ having different configurations
→ machine independent.

② Structured Language

③ Case Sensitive.

④ Pointers are used.

⑤ Dynamic memory allocation.

⑥ Compiler & processing is faster.

Algorithm → Set of instructions to perform a particular task.

→ Uses pseudocode, means using natural language.

Algorithm may vary from user to user.

→ Algorithm cannot be executed. Algorithm has to be written as a program using some programming language for execution.

Adding 2 numbers

- ① Start
- ② Read 2 nos as A & B
- ③ Add A & B & store result in C
- ④ Display C
- ⑤ Stop

Program

```
#include <stdio.h>
```

```
main() {
```

```
    int a = 10, b = 20;
```

```
    int c;
```

```
    c = a + b;
```

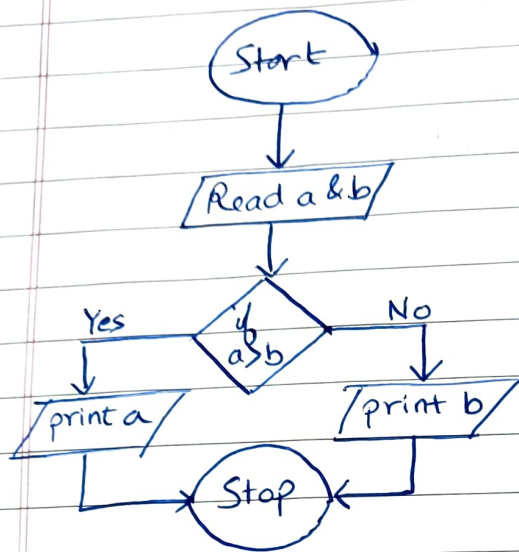
```
    printf("The sum of %d & %d is %d", a, b, c);
```

```
}
```

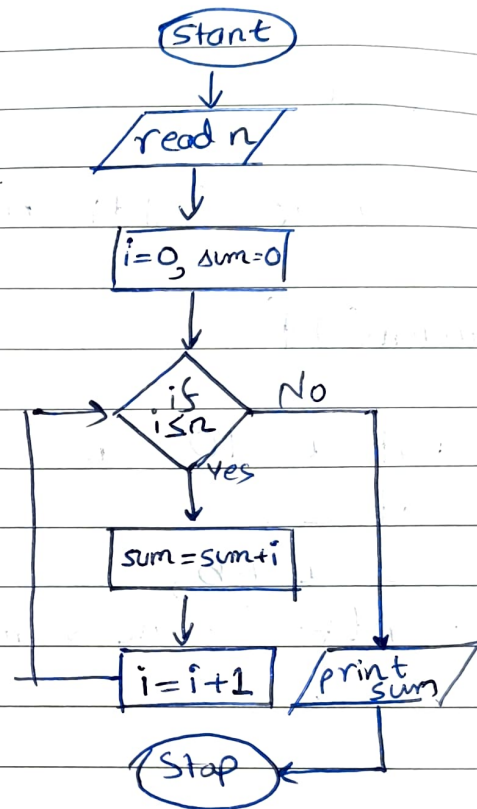
Q Write algorithm to find average of 3 numbers.

- ① Start
- ② Read 3 no, A, B, C
- ③ Add A, B, C, store in D
- ④ Divide D by 3, store in E
- ⑤ Display E
- ⑥ Stop

Given two numbers, find greatest of them.



Find sum of 'n' numbers starting from 0.



Characteristics of algorithm

- 1) It takes some input and returns the desired output.
- 2) Algo must be finite, terminate after particular steps.
- 3) Number of instructions should be finite.
- 4) Clear and unambiguous.
- 5) Robust; applicable to all input/output.

C Programming basics

Identifiers: name for identifying variable, array, function, letters, numbers, underscore.

Keyword: predefined words which has been already defined. There are 32 keywords.

Var $\rightarrow$ Name given to a memory location where data is stored

Datatype $\rightarrow$ int

Steps $\rightarrow$ 1) declaration
2) initialization

int $\rightarrow$ 2 bytes

float $\rightarrow$ 4 "

char = 1 "

double = 8 "

User Inputs

`scanf("specifier", &var)`

Ex: `int a, b, c;`

`printf("Enter 2 numbers");`

`scanf("%d %d", &a, &b);`

`c = a + b;`

`printf("Sum is %d", c);`

Type Conversion

1) Implicit $\rightarrow$ automatically by compiler.

2) Explicit $\rightarrow$ User.

Implicit, when in an exp, there is more than one datatype

Lower $\rightarrow$ higher

char $\rightarrow$ int $\rightarrow$ float $\rightarrow$ double

Ex

`int a = 7;`

`float b = 9.5;`

`int c = a / b; returns 1`

`float`

`double d = a / b; returns 1.9000`

Explicit (Type Casting)

(float) a
(int) a

```
{ float a = 20.8, b = 10.5;  
  int c = a + b;  
  { int d = (int) a + (int) b;  
    (int)(a + b)  
    (int) a + b
```

c $\Rightarrow$ 31

d $\Rightarrow$ 30

Operators $\rightarrow$ symbols for performing mathematical & logical ops.

① Arithmetic operators

+, -, %, /, *

$\downarrow$
int

a % b

$\downarrow$

For %

Output depends on sign of a.

② Relational operators

returns either true / false.

<, <=, >, >=, ==, !=

③ Assignment Operator

=, +=, -=, *=, /=, %=

④ sizeof() operator : size of the variable, in integer (bytes)

Logical operator → used to combine relational exp.

AND & & → op1 & op2

OR || → op1 || op2

NOT !

op1	op2	& &		!
T	T	T	T	0 → 1
T	F	F	T	1 → 0
F	T	F	T	
F	F	F	F	

Conditional op statement

if ~~else~~, else, ~~if else~~ if, switch.

else is optional,

Q) Write a program, take a number & check if number is even or not.

⇒ #include <stdio.h>

```
void main() {  
    int a;  
    printf("Enter the number: ");  
    scanf("%d", &a);  
    if (a % 2 == 0) {  
        printf("Number is even");  
    }  
    else {  
        printf("Number is odd");  
    }  
}
```

Nested if-else

```
if (condition) {  
    if (condition-2) {  
        }  
}  
else {  
    if (condition-3) {  
        }  
}
```

Increment & decrement op

++, --

prefix

++a

postfix

a++

++a → add 1 & then assign

a++ → Assign then add 1

x = 10;

c = ++x; → c → 11, x = 11

c = x++; → c → 10, x → 11

Ternary operator (?)

(condition) ? a₁ : a₂

 ↑ ↑
 T F

Ex) a = 10, b = 20, c = 30

!((a < b) || (c > b)) ? a + b : a - b

 F T

 ↑ ↑
 F T

a - b → 10 - 20
 = -10

Bitwise Operator $\Rightarrow$ bit level operations.

OR			
AND	&		
XOR	^		
NOT	~		
Left Shift	<<		
Right Shift	>>		

$$\begin{array}{cccc}
 & 3 & 2 & 1 & 0 \\
 & 1 & 0 & 1 & 0 \\
 1 \times 2^3 & + & 0 \times 2^2 & + & 1 \times 2^1 & + & 0 \times 2^0
 \end{array}$$

XOR
 $A \wedge B$

A	B	Result
0	0	0
1	0	1
0	1	1
1	1	0

NOT
 $A \sim B$

A	Result
0	1
1	0

Shift operators

<<, >>

$\Rightarrow 10 \ll 2$
 $\Rightarrow$ no. of bits for shifting

100110110 << 2
 $\Rightarrow$ 11011000

$\Rightarrow$ fill the empty space with zero

Constant

Variable having fixed value

• Numeric $\rightarrow$ int $\begin{cases} \rightarrow$ octal
 $\rightarrow$ decimal
 $\rightarrow$ float
 $\rightarrow$ hexa

124 $\rightarrow$ decimal
0124 $\rightarrow$ octal
0x214 $\rightarrow$ hexa

• char

const

#define

Switch Case

block of code that executes depending on the value of a var.

switch (exp)

{

case <'label':

}

Example

```
int n = 3;
```

```
switch(n) {
```

```
    case (1):
```

```
        printf("A");
```

```
        break;
```

```
    case (2):
```

```
        printf("B");
```

```
        break;
```

```
    case (3):
```

```
        printf("C"); break;
```

```
    case default:
```

```
        printf("No value");
```

```
        break;
```

```
}
```

Output:

C

$a=50, b=70;$

$$z = \underbrace{(a \% 3 < 3)}_1 + \underbrace{b * 3}_1 \& \& \underbrace{a + b}_1 = 120 \& \& 70 = b > 7$$

$$z = ((\underbrace{a \% 3 < 3}_1) + \underbrace{b * 3}_1) \& \& (\underbrace{a + b}_1 \& \underbrace{120}_0 \& \underbrace{70}_1 = b)$$

Operator precedence

Arithmetic $\rightarrow$ Relational $\rightarrow$ Logical $\rightarrow$ Assignment.

Loops

for loop,
while loop,
do-while loop,

For loop

(boolean)

for (initial value; condition; increment/decrement) {

}

Ex: for ($i=0$; $i \leq 10$; $i=i+1$) {
 printf("ABC");
}

Q) Write a program to count 100 to 50

```
int i;  
for (i=100; i >= 50; i--) {  
    print("%d", i);  
}
```

Q Print even number from 100 to 50 in reverse order.

```
for (i=100; i>=50; i--) {  
    if (i%2==0) {  
        printf("%d", i);  
    }  
}
```

~~loop~~

Q Nested for

```
int i, j;  
for (i=1; i<=3; i++) {  
    for (j=1; j<=5; j++) {  
        printf("%d %d", i, j);  
    }  
}
```

Iterations: 15

While Loop

```
while (i<=5)  
{  
    printf("%d", i);  
    i=i+1;  
}
```

Write a program in while loop to find average of numbers from 1 to 10.


```
#include <stdio.h>
```

```
main() {
```

```
    int i = 1; int sum = 0;
```

```
    while (i <= 10) {
```

```
        sum = sum + i;  
        i = i + 1;
```

```
    int average = sum / 10;
```

```
    printf("%d", average);
```

```
}
```

do while

~~do~~ condition check at bottom.

```
do {
```

```
    do
```

```
    while (<cond>)
```

Ex

```
int i = 1;
```

```
do
```

```
{
```

```
    printf("%d", i);
```

```
    i = i + 1;
```

```
}
```

```
while (i > 10)
```

It executes atleast 1.

break statement

Use to exit the loop.

```
int i;  
for (i=1; i<10; i=i+1) {  
    printf("%d", i);  
    if (i==5) {  
        break;  
    }  
}
```

Output

1 2 3 4 5

Continue statement

skips the rest of the loop. statement.

Array: collection of values having same datatype stored at contiguous memory locations.

Static data structure:

↓ datatype

```
int a[5]  
  ↓      ↗ array size  
array name
```

elements	10	20	30	40	50
Index:	0	1	2	3	4

$\text{int } a[5] = \{1, 2, 3, 4, 5\}$

or

in $a[5]$

$a[0] = 1,$
 $a[1] = 2,$

address		index
1000	10	0
1002	20	1
1004	30	2
1006	40	3
1008	50	4
1010		

Base address $\rightarrow 1000$

Name of array gives the base address.

To get address:
 $\text{base address} + \text{sizeof data type} \times \text{index}$

Addr of n^{th} element:

$$\text{b. address} + \text{sizeof data type} \times (n-1)$$

Q) Find address of ^{array} element at index 15, whose base address is 2000 and elements are of float data type

$$\begin{aligned} & \rightarrow 2000 + 4 \times 15 \\ & \rightarrow 2060 \end{aligned}$$

Q) Write a program where you insert 8 elements into an array using user input & then find the average of these elements.

```
#include <stdio.h>
```

```
main() {
```

```
main() int a[8];
```

```
for (int i=0; i<=8; i++) {
```

```
    printf("Enter element %d", i);
```

```
    scanf("%d", &a[i]);
```

```
int sum;
```

```
for (int i=0; i<=8; i++) {
```

```
    sum = sum + a[i];
```

```
}
```

```
printf("Average is %d", sum/8);
```

```
}
```

2D Array

Matrix

multiple rows 2 columns

a[0][0]	a[0][1]
a[1][0]	a[1][1]

int a[4][3]

↓ ↓
rows columns

memory: 2×6
= 12 bytes

printf("%d", sizeof(a));

>> 12 bytes

int a[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

8) To print columns

```
for (i=0; i<3; i++) {  
    for (j=0; j<2; j++) {  
        printf("%d", a[j][i]);  
    }  
}
```

To print rows

```
for (i=0; i<2; i++) {  
    for (j=0; j<2; j++) {  
        printf("%d", c[i][j]);  
    }  
}
```


Q write a program to find transpose of a matrix,

Ans #include <stdio.h>

main () {

int m[3][3], r[3][3];

for (int i=0, i<3, i++) {

for (int j=0, j<3, j++) {

scanf ("%d", &m[i][j]);

r[j][i] = m[i][j];

}

}

for (int i=0, i<3, i++) {

for (int j=0, j<3, j++) {

printf ("%d", r[i][j]);

}

}

}

Functions

A block of code to perform a particular task.

```
int function (int n, int y) {
```

...

return x

```
}
```

→ Avoiding rewriting of code.

Structure

return type

function (data type argument)

If there is no return value, we write void as function datatype.

By default the return type will be ~~of~~ int.

Test (float n) ^{→ called function.}

^{→ formal argument}

test(a) ^{→ actual argument}
calling function.

Number of formal arg = Number of actual arg.

Type of function

- 1) without arg & without return type, value
- 2) without arg & with return type, value
- 3) with arg & with return type, value
- 4) with arg & without return type, value

~~void test(a, b)~~

Ex: ~~without~~ ^{if a} with arg & without return

```
void test (int a, int b) {  
    if (a > b) {  
        printf("%d", a);  
    }  
    else {  
        printf("%d", b);  
    }  
}
```

```
main() {  
    int a = 1; b = 2;  
    test(a, b);  
}
```

Example

```
int arr[10]; {  
arr[0] = 1;  
arr[1] = 2;  
}
```

a. —

```
int arr[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

test(arr) → calling function

Scope of variable (region within which the var
got its existence)

1) Local var

2) Global var

3) Static var

Local Variable

Variable within a function. They can't be
accessed used in other functions.

Global Variable

Variable outside function, as they can be
used throughout the program. The value of
global variable can be changed.

Static Variable

Declared within a function but it exists after the
function.

```
main() {
```

```
    test();
```

```
    test();
```

```
    test();
```

```
void test() {
```

```
    static int i = 0;
```

by default static is
initialized to 0

WAP to add & sub two numbers using 2 diff fun

```
#include <stdio.h>
```

```
int add(inta, intb) {  
    return a + b;  
}
```

```
void sub(inta, intb) {
```

```
    printf("%d", a - b);  
}
```

```
main() {
```

```
    x = 1;
```

```
    y = 1;
```

```
    printf("%d", add(x, y));
```

```
    // printf("%d", sub(x, y));
```

```
    sub(x, y);  
}
```

Recursion

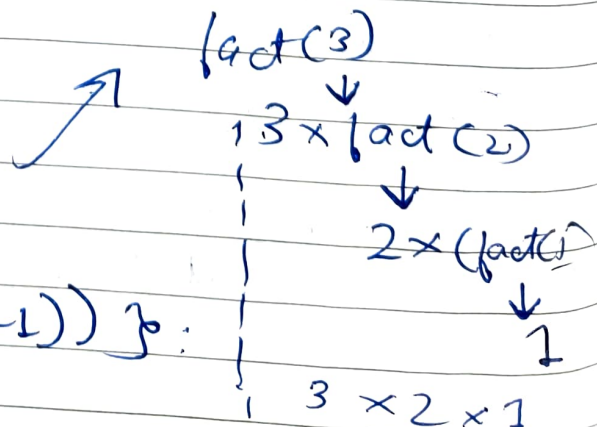
When a function calls itself until it reaches a stopping condition.

Ex: function

```
int fact(int n) {  
    if (n <= 1) {  
        return 1;
```

```
    else {
```

```
        return (n * fact(n-1));  
    }
```



String

array of characters which ends with a null character
'\0'

char test[30] = "Hello"; ^{↗ 6 bytes}
H + e + l + l + o + \0

'\0' is used to differentiate b/w array of characters & string.

```
printf("%s", a);  
scanf("%s", a); // Not &
```

&a[0] = a

#include <string.h> → new header

fgetc → scanf

puts → printf

```
fgetc(a, sizeof(a), stdin);  
puts(a);
```

~~strlen~~

strlen()

strcpy()

strcat()

- 1) Explain about recursion & its different types.
Write programs to explain each type of recursion.

Recursion is mainly of 2 types:

① Direct recursion: When a function calls itself, there are different ways to call the function in this type:

② Tail recursion: The function is called again in the last statement.

```
Ex: #include <stdio.h>
void func(int n)
{
    if (n > 0) {
        printf("00 %d", n);
        func(n-1);
    }
}

int main() {
    func(10);
    return 0;
}
```

③ Head recursion: The function is called again in the first statement (after compare statement).

Ex:

```
#include <stdio.h>
void func(int n)
{
    if (n > 0) {
        func(n-1);
        printf("%d", n);
    }
}

int main() {
    func(10);
}
```

③ Tree recursion: recursive function calling itself for more than one time.

```
#include <stdio.h>
void func(int n)
{
    if (n > 0) {
        func(n-1);
        func(n-1);
        printf("%d", n);
    }
}

int main() {
    func(10);
}
```

④ Nested recursion: recursive function which is passed a parameter of that recursive function.

```
#include <stdio.h>
void func(int n)
{
    if (n > 0) {
        return func(func(n-1));
    }
}
```



```
#include <stdio.h>
int func(int n) {
    if (n > 100)
        return n - 10;
    return func(func(n + 11));
}

int main() {
    func(95)
}
```

① Linear Recursion: a recursive function which is called only once.

§

② Indirect recursion: In this recursion, there can be more than one functions and they are calling one another in circular manner.

```
#include <stdio.h>
void funcA
void funcB(int n);
void funcA(int n) {
    if (n <= 0) {
        return;
    }
    funcB(n - 1);
}

void funcB(int n) {
    if (n <= 0) {
        return;
    }
    funcA(n - 1);
}

int main() {
    funcA(5)
}
```


2) Illustrate on the following string handling function:

@strlen() @strcat() @strcpy() @strcmp()
@strrev()

Write a program which includes these function.

Ans: @strlen(): calculates the length of string
Ex: strlen("Hello") → 5

@strcat(): joins two string together
Ex: strcat("A", "B") → AB

@strcpy(): copies one string to another
Ex: char dest[50];
char *src = "Hello";
strcpy(dest, src);
printf("String %s", dest);

@strcmp(): compares two string
strcmp("Hello", "Hello") → true/1

@strrev(): reverses the string letters
strrev("Hello") → olleH

Program

```
#include <stdio.h>
```

```
int #include <string.h>
```

```
void main() {
```

```
    char str1[50] = "Hello"
```

```
    char str2[50] = "World"
```

```
    char str3[50];
```

```
    printf("Length of str1: %d", strlen(str1));
```

```
    printf("Adding str2: %s", strcat(str1, str2));
```

```
    printf("Copied to str3: %s", strcpy(str3, str1));
```

```
    printf("Comparing str1 & str2: %d", strcmp(str1, str2));
```

```
    printf("reversing str1: %s", strrev(str1));
```

```
}
```

3) Summarise on 2-D arrays. Write a program to do matrix multiplication considering the compatibility factor where matrices can be of any size.

Ans: 2-D arrays are simply arrays of arrays. used to store data in a tabular form. They are particularly useful for representing matrices and grids.

Program

```
#include <stdio.h>
```

```
void main() {
```

```
    int mat1[10][10];
```

```
    int mat2[10][10];
```

```
    int i, j, k;
```

```
    int rows, cols;
```

```
    printf("Enter number of rows");
```

```
    scanf("%d", &rows);
```

```
    printf("Enter number of columns");
```

```
    scanf("%d", &cols);
```

```
    printf("\nEnter elements of first matrix:\n");
    for (i = 0; i < rows; i++) {
```

```
        for (j = 0; j < cols; j++) {
```

```
            printf("Enter element at [%d][%d]", i, j);
```

```
            scanf("%d", &mat1[i][j]);
```

```
        }
```

```
    }
```


printf

```
printf("\nEnter element of second matrix:\n");  
for (i=0; i < rows; i++) {  
    for (j=0; j < cols; j++) {  
        printf("Enter element at [%d][%d]:", i, j);  
        scanf("%d", &mat2[i][j]);  
    }  
}
```

```
int result[10][10];  
for (i=0; i < rows; i++) {  
    for (j=0; j < cols; j++) {  
        result[i][j] = 0;  
    }  
}
```

// matrix multiply

```
for (i=0; i < rows; i++) {  
    for (j=0; j < cols; j++) {  
        for (k=0; k < cols; k++) {  
            result[i][j] += mat1[i][k] * mat2[k][j];  
        }  
    }  
}
```

// show result

```
printf("Resultant matrix\n");  
for (i=0; i < rows; i++) {  
    for (j=0; j < cols; j++) {  
        printf("%d\t", result[i][j]);  
    }  
    printf("\n");  
}
```

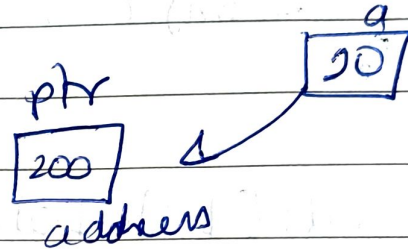
}

Pointers

It is a variable which stores the address of another variable.

$\&$ $\rightarrow$ address operator.

```
int a = 10;  
int *ptr;
```



Addresses are hexadecimal numbers.

To print $\rightarrow$ $\%d$ returns address

```
int *ptr  
↓
```

datatype of the value stored.

$\text{printf}(\%d, *ptr)$ $\rightarrow$ return value stored in pointer

Call by reference

```
void test (int *a) {  
    printf ("%d", *a);  
    *a = *a * 10;  
    printf ("%d", *a);  
}
```

```
main() {  
    int n = 10;  
    printf ("%d", n);  
    test (&n);  
    printf ("%d", n);  
}
```

Output: 10 10 100 100

~~test(*arr)~~

Passing an array to function is always call by reference.

Pointer Arithmetic

```
int a[4] = {1, 2, 3, 4}, *n,
```

```
n = &a[0]
```

```
printf ("%d", n+1);
```

a[10] 1000

Output

$n+1 \rightarrow 1000 + 2 \times \text{sizeof}(\text{datatype})$
 $= 1002$

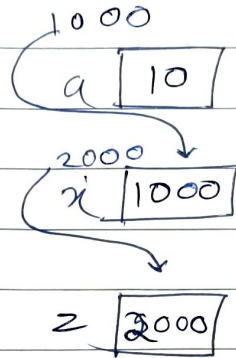
$\text{address} + i * \text{sizeof}(\text{datatype})$

Double pointer

↳ var that stores address of another pointer.

**

```
int a = 10;
int *n = &a;
int **z = &n;
```



```
* z = 1000
* n = 10
```

```
** z = 10
```

User defined datatype

1) typedef 2) enum

typedef : used to assign a new name to predefined datatype.

```
typedef int integer;
```

↳ integer a = 10; ✓

```
typedef long int li
```

↳ li b = 373;

enum → assigning names to integer constants.

```
enum day (Sun, Mon, Tue, Wed, Thu, Fri, Sat);
```

0 1 2 3 4 5 6

```
day today;
```

```
today = Wed;
```

```
printf ("%d", Wed today);
```

↓

or

```
enum day (Sun=10, Mon=11, ...);
```

```
scanf ("%u", &today);
```

* Example

Use enum to specify two genders, male, female.

```
#include <stdio.h>
```

```
main() {
```

```
    enum gender (male, female);
```

```
    gender unknown;
```

```
    scanf ("%u", &unknown);
```

```
    if (unknown == 0) { // (gender == male) {
        printf ("You are male");
    } else {
```

```
        printf ("You are female");
    }
```



```
for (i = Sun; i <= Sat; i++) {
```

* Errors

- ① Syntax Error → Compile time, code errors.
- ② Runtime Error → occurs when prog is running.
- ③ Logical Error → mistake in the logic part of the program (wrong output)
- ④ Linking Error → difficulty linking to header files/library
- ⑤ Semantic Error → when compiler can't understand the statements.

Structure

User defined datatype
Used for keeping records.

Creating the datatype -
↓ datatype ↓ name of structure.

```
struct students {  
    char name[50];  
    int id;  
    int age;  
};
```

} members of structure.

Declaration.

```
struct student s;  
    ..
```

```
} s1, s2;
```

or

```
def main()
```

```
struct students s1, s2;
```

- * Written before main()
- * Memory not occupied in defining struct

Initializing values to struct variable.

s1.name = "abc";

s1.age = 20

s1.id = 123

dot(.)

printf("%s", s1.name);

scanf("%s", s1.name);

↓ array of characters
don't require &

scanf("%d", &s1.id);

Memory allocator → sum of variables
size of.

Union structure.

char na[50];	}	struct		union
int age;		54		80
int id;				

↓

Same as struct, but memory allocated will be size of highest variable.

Structure of C program

① Documentation

↳ comments

single line → //

multiple line → /*....*/

(optional)

② Link Section

↳ define the header files, libraries

③ Definition

↳ define constant. #define or const.
(optional)

④ Global declaration

↳ define global variables

⑤ Main function

↳ compiler starts from main section.

↓
→ Declaration (optional)

→ Execution

(compulsory section)

⑥ Sub program

↳ user defined function.

(optional)

Execution process of C program

① .c → source file (test.c)

It can't compile pre-processor directives.

↓
statements which begins with (#)
or function

② Pre-processor → it replace statements with their meaning / source code, test.i

③ Compiler → returns assembly file → test-asm code written in assembly language

④ Assembler ^{returns} → object file (.obj)
(Not executable file)

⑤

⑤ Linker → links the libraries or
header files to the obj files
test.exe
*(Executable file)

⑥ Loader → loads the file in memory (RAM)
Then CPU executes the code.